V1.5 — Cheap Semantic Worker and Selective Escalation

Reflex Engine, experiment V1.5 · OUTCOME E · 2026-09-01

A deterministic worker can describe a query cheaply. It cannot answer one safely.

On 800 sealed real-language items the full V1.5 pipeline certified **5 answers, 3 correct, 2 wrong** — a 0.63% deterministic coverage at a **40% wrong-fire**. The pre-registered safety threshold is 2%. Rung E fires, and it outranks every economic question the version was built to ask.

---

1. Verdict against the frozen ladder

	condition	result
F	no outcome met, or Tier A regressed, or V1.3/V1.4 stop reproducing	**not triggered** — Tier A certified, V1.3 reproduces, all tests pass
E	safety or semantic integrity fails: wrong-fire above 2% on any sealed arm	**TRIGGERED — 40% on A4 and A6**
D	coverage below 15%, so the energy ceiling stays uninteresting	would also fire — 0.63%
C	helps small models but not opus	not reached
B	advantage only after amortization	not reached
A	≥20% energy saving with safety and coverage intact	not reached

OUTCOME E. Both thresholds failed, and safety outranks economics by the frozen priority. The version does not reach the question it was designed to answer.

---

2. Sealed results

800 items, `s3_sealed`, **zero model calls**, deterministic throughout.

arm	certified	correct	wrong	coverage	wrong-fire	ms/query
A2 worker only	0	0	0	0.00%	—	6.14
A3 + substrate	0	0	0	0.00%	—	6.37
A4 + path/execution	5	3	**2**	0.63%	**40.0%**	6.27
A6 full V1.5	5	3	**2**	0.63%	**40.0%**	6.27
A7 no relational structure	0	0	0	0.00%	—	6.24

Wrong-fire Wilson upper bound on A6: **76.9%**. At five served answers nothing can be claimed in either direction, which is itself the point — the pipeline does not serve enough to be measured.

Escalation fraction **93.25%**, distributed as:

```
name_these_quantities   507
concept_disambiguation  222
name_the_target          14
conflict_resolution       2
whole_query               1
```

95%+ of escalations name a field rather than re-asking the query.

That is the brief's §6 mechanism working exactly as specified — and by the plan's D-5 it buys safety, not energy.

---

3. The economics, settled by arithmetic before measurement

The plan established the identity in §2.1, before any sealed run:

```
E_v15 ≈ f_escalate × E_llm + E_worker + E_engine
      ≈ f_escalate × E_llm            (worker ≈ 0.5% of one call)

saving ≈ 1 − f_escalate = deterministic coverage
```

The worker costs **6.3 ms** against a measured pure-LLM call of **64.9 GPU-J** — about half a percent. So the saving is, to within rounding, just the fraction of queries answered without a model.

Measured: 0.63%. The projected end-to-end energy saving is therefore **0.63%**, against a 20% threshold.

No energy run was needed to establish this and none was performed. The identity makes the measurement redundant: an arm that answers 5 of 800 queries deterministically cannot save 20% of anything, whatever the meter says. Reporting a joule figure here would add precision to a conclusion that does not need it.

What the brief expected, and what is true

The brief's §6 reasons that escalating the smallest unresolved unit is cheaper than escalating the query. **It is not**, on this baseline, and the plan froze that as D-5 before measuring. The pure-LLM comparator already asks for a bare number at `max_tokens=24` — the cheapest call available. Selective escalation saves by **not calling**; a partial escalation costs about what a whole one does.

What it buys instead is real and worth keeping: the model is asked to NAME a field, and the reply is validated against the text rather than

trusted as a value. The engine never receives an answer it cannot check.

---

4. The failure, and why guards did not fix it

The executor's first version defaulted to `add` whenever a question asked for a total and no operator had been established. On the dev split that produced **7 wrong answers in 16 served — 43.75%**:

```
"Jake has six fewer peaches than Steven. Steven has 13."
     6 + 13 = 19        gold 7
"Steve gave him 16 more. Now Todd has 54 pieces."
     16 + 54 = 70       gold 38    (a reverse problem)
"7 flowers each representing 7 colours ... 70 bouquets"
     walked to 84       gold 490   (a rate)
```

Four guards were added, each for a documented defect: no default operator; a rate/grouping guard; non-negativity; and a superlative guard for optimisation and GCD problems (V1.3's D10 class). On dev they worked — **4 served, 4 correct, 0 wrong**.

On sealed data they did not.

```
"A waiter had 47 customers. If 41 left and he got another 20..."
     47 + 41 + 20 = 108     gold 26     ("left" never became a subtraction)
"There are 9,300 pennies in a cup. What is the total dollar amount?"
     walked to 18,600       gold 62     (no unit conversion)
```

Same class as the dev failures: the walk composed events whose composition it had not established. **The guards were patches fitted to the dev split, not rules that generalise** — dev 0% wrong-fire, sealed 40%. That gap is the most useful thing this version measured, and it would have been invisible without the split.

---

5. What did work

The section-4 distinction test passes. Four sentences a flat representation cannot tell apart:

```
John has 5 apples                  possession
John gives 5 apples to Sarah       transfer john→sarah
Sarah gives John 5 apples          transfer sarah→john
John had 5 apples before giving    possession(0) + transfer(1)
  Sarah 2 apples                     john→sarah
```

The decisive pair is prepositional versus ditransitive: identical tokens, opposite direction, and word order carries the whole meaning. Two paraphrase pairs are held together as negative controls, so the result is not an artifact of the indexer emitting unique signatures.

The packet is cheap and honest. 6.3 ms per query, 0 unverifiable spans across 400 items, and a typed verdict naming what is missing: MISSING_CONCEPT 55.8%, AMBIGUOUS 24.5%, COMPLETE 15.2%, MISSING_TARGET 4.0%, CONFLICTING 0.5% *(dev figures)*.

Escalation is genuinely selective. 95%+ of unresolved queries name a specific missing field rather than falling back to the whole query.

---

6. Two findings that outlive the version

B25 — the namer fabricated physics kinds. `nlpw10`'s fallbacks almost never fire on the generator universe and fire constantly on real text, returning a confident wrong kind instead of UNKNOWN. `payback_months` **was assigned 131 times across 400 real queries** — to snow depth, schoolchildren, touchdowns. This is worse than a coverage miss: UNKNOWN abstains, but `payback_months` carries a dimension, so the dimensional gate engages, passes, and certifies nonsense. It is V1.3's D5 in its most damaging form, and it also explains V1.4's false conflicts. Fixed by regrounding through evidence-backed naming; 206 of 1,068 quantities were fabricated.

B26 — the conflict rule fired on lists. *"Seven red apples and two green apples"* looked like one entity holding two values, because the everyday lexicon names every countable `count`. A genuine conflict is the text **asserting twice**, which takes two clauses. CONFLICTING fell 22.5% → 0.5%.

---

7. A discipline failure of mine

I developed against a sealed set. The packet, index and executor were all built while measuring on the first 400 items of S3 — a sealed evaluation set — with their answers in view.

Caught mid-build and resolved rather than at write-up: S3 was split at 400, items 0–399 declared contaminated and demoted to `s3_dev`, items 400–1199 sealed as `s3_sealed` (800 items, sha256 recorded, never touched until the single run in §2).

The cost is real and is not a rounding detail. 400 of 1,200 fresh real-language items are spent, and the corpus has no more — V1.3 and V1.4 consumed the rest. **This version's sealed half is smaller than planned because I was careless**, and every interval in §2 is wider for

it.

It did, however, produce the version's best finding: without the split,

the dev-tuned guards would have been reported at 0% wrong-fire.

---

8. Hypotheses

	claim	result
H1	cheap worker builds useful packets for a substantial fraction	**PARTIAL** — 15.2% complete, but only 0.63% answerable
H2	substrate beats a flat representation on integrity	**CONFIRMED** — A7 (no structure) serves 0; the section-4 test passes
H3	selective escalation costs less than full-query reasoning	**FALSIFIED as framed** — it saves by avoiding calls, not by cheaper calls (pre-registered D-5)
H4	typed handoffs preserve the safety advantage	**NOT EXERCISED** — no escalation was executed
H5	packet reuse amortizes interpretation	**NOT RUN**
H6	small manager operates a large-model substrate	**NOT RUN**
H7	opus benefits economically from reflexes	**NOT RUN**
H8	advantage grows with complexity/repetition	**NOT RUN**
H9	explicit UNKNOWN reduces wrong-fire without destroying coverage	**HALF** — abstention is the dominant behaviour; the coverage that survives is 0.63% and still 40% wrong
H10	Path Manager composes reflex→LLM→reflex	**NOT RUN**
H11	relational structure generalises better than fixed labels	**CONFIRMED for structure, FALSIFIED for guards** (§4)
H12	lower end-to-end J/correct than pure LLM	**FALSIFIED** — 0.63% projected saving against a 20% bar

---

9. What V1.5 does not claim

- **No model arms were run.** Haiku, sonnet, opus and llama arms, the manager-transfer matrix and the amortization workload are all unrun.

The deterministic core failed its safety threshold first, and running

model arms on a pipeline that certifies 5 of 800 would measure

nothing.

- **No energy measurement was taken**, by design — §3 explains why the identity makes it redundant at this coverage, not why it was skipped.
- **No amortization curve**, no break-even query, no tier breakdown.
- The 15.2% packet-completeness figure is a **dev** number and an upper bound on coverage, never coverage itself.

10. The remaining bottleneck, named precisely

It is not extraction. The worker describes a query in 6 ms with zero unverifiable spans and names what it is missing.

It is not representation. The substrate distinguishes all four of the brief's cases including direction and order, and the arm without it serves nothing.

It is composition. Knowing that a story contains 47, 41 and 20, that someone *left* and someone *got another*, is not knowing whether the answer is `47−41+20` or something else. Every wrong answer in this version — dev and sealed — is the engine composing events whose composition it had not established. Four guards reduced that on dev and did not generalise, because they encode which compositions are *wrong* rather than establishing which is *right*.

11. V1.6 recommendation

Stop trying to make the deterministic path answer. On this evidence it cannot do so safely at any useful coverage, and three versions have now established that from different directions: V1.3 read language and got 1.44% wrong, V1.4 built the representation and served nothing, V1.5 added a cheap reader and served 5 of 800 at 40% wrong.

The asset that has survived every version is the **abstention** — the system knows what it does not know, and says so specifically. The packet names its missing field 95%+ of the time. That is worth something on its own, and it is worth more than a coverage number bought at 40% wrong-fire.

The honest next question is not *"can the engine answer?"* but **"is a system that reliably says what it cannot determine worth deploying in front of a model that always answers?"** That is a different product and a measurable one, and this arc has accidentally built most of it.

12. The one-paragraph version

V1.5 asked whether a cheap deterministic worker could answer enough natural language without a model to lower whole-system energy. The

worker is cheap — 6 ms, half a percent of one LLM call — and the substrate distinguishes stories a flat representation cannot. But the pipeline certified 5 answers in 800 sealed items and got 2 of them wrong, so the safety threshold fails at 40% against a 2% bar and the energy saving is 0.63% against a 20% bar. The bottleneck is neither extraction nor representation but **composition**: knowing what a story contains is not knowing what to do with it, and four guards that achieved zero wrong-fire on dev achieved 40% on sealed data. The most valuable thing measured was that gap, and it was only visible because the split existed — which it did only because I noticed mid-build that I had been developing against sealed data.